

Java I/O Notes

IOException, Streams, Readers & Writers, and the File Class

IOException

An IOException is any unexpected problem the JVM encounters while attempting to run a program. When an IOException is thrown, it means that whatever is throwing the exception (perhaps a try{}-catch block that reads data from a file) can throw an IOException — for example if the file is not found, corrupted, etc., or when the file is otherwise unable to be read, or any other of a list of issues that can occur with the IO package and its extensions.

Streams

In its most basic form, a stream simply represents a sequence of data (bytes or Unicode characters) in some sort of a sequential queue. Java programs perform I/O through streams. A stream is an abstraction that either produces or consumes information. A stream is linked to a physical device by the Java I/O system.

Java implements streams within class hierarchies defined in the java.io package. The same I/O classes and methods can be applied to different types of devices.

Java defines two types of streams:

Byte Streams

Byte streams provide a convenient means for handling input and output of bytes. Byte streams contain binary data — this is useful for things such as reading and writing to files. Just imagine opening up an image file in a text editor; that is a good representation of byte data.

Example: using a byte stream (e.g. InputStream) and typing '■' (a runic character) exceeds the 256 limit of 8-bit characters, so it will show some other value. If we type '■' in a char-stream such as InputStreamReader, it will show '■' correctly.

```
OutputStream os = System.out;  
os.write('■'); // will not show anything (or garbage) cuz range exceeded. ■ is Unicode.
```

Character Streams

Character streams provide a convenient means for handling input and output of characters. They use Unicode and, therefore, can be internationalized — good candidates for things like keyboard input and console output. Also, in some cases, character streams are more efficient than byte streams.

Byte streams are defined by using two class hierarchies. At the top are two abstract classes: InputStream and OutputStream. Each of these abstract classes has several concrete subclasses that handle the differences among various devices, such as disk files, network connections, and even memory buffers. The abstract classes InputStream and OutputStream define several key methods that the other stream classes implement. Two of the most important methods are read() and write().

Character streams are defined by using two class hierarchies. At the top are two abstract classes: Reader and Writer. Two of the most important methods are read() and write().

The Predefined Streams

- System.out refers to the standard output stream. By default, this is the console.
- System.in refers to standard input, which is the keyboard by default.
- System.err refers to the standard error stream, which also is the console by default.

System.in is an object of type InputStream; System.out and System.err are objects of type PrintStream. These are byte streams.

NOTE: Names of character streams typically end with Reader/Writer, and names of byte streams end with InputStream/OutputStream.

Reader Classes

InputStreamReader Class

Extends Reader — an abstract class for reading character streams.

```
int read() // returns the character read, as an integer in the range 0 to 65535
(0x00-0xffff),
// or -1 if the end of the stream has been reached
```

Primarily it is used to convert byte streams to character streams. java.io.InputStreamReader has several overloaded constructors, each taking an InputStream (like System.in, or FileInputStream) as the first parameter.

The InputStreamReader class has a method called close() that will close the stream and release any system resources associated with it. The close() method should always be called once you are done with the input stream.

In Java 7 a new feature called try-with-resources was introduced. Specifically, a resource is an object that must be closed after the program is finished with it. The try-with-resources statement ensures that each resource is closed at the end of the statement. How do we know if a class is a resource? Simple — if it implements java.lang.AutoCloseable, the class can be considered a resource.

try-with-resources

The try-with-resources statement is a try statement that declares one or more resources. A resource is an object that must be closed after the program is finished with it. The try-with-resources statement ensures that each resource is closed at the end of the statement. Any object that implements java.lang.AutoCloseable, which includes all objects which implement java.io.Closeable, can be used as a resource.

FileReader Class

The purpose of the FileReader class is to simply read character-based files. The FileReader class implements AutoCloseable, so we can use the try-with-resources type exception handling. It is also interesting to note that the FileReader class does not define or override any public methods, so it inherits all of its methods from its

superclass `InputStreamReader`; `InputStreamReader` is a subclass of `Reader`, which is a subclass of `Object` — and that describes the class hierarchy.

Object → Reader → InputStreamReader → FileReader

BufferedReader Class

```
BufferedReader(Reader inputReader)
```

It extends `Reader`, hence has its own `read()`, which is the same as `InputStreamReader`'s. The `BufferedReader` class is used to read text from a character stream. The `BufferedReader` class has the same `read()` method that both `InputStreamReader` and `FileReader` use to read a single byte at a time. The `BufferedReader` class introduces a method named `readLine()`, which will read an entire line of text — a huge improvement. The `BufferedReader` class implements `AutoCloseable` so we can use the try-with-resources type exception handling.

To obtain a character-based stream that is attached to the console, wrap `System.in` in a `BufferedReader` object:

```
InputStreamReader(inputStream)
```

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
```

NOTE: `new InputStreamReader(System.in)` converts a byte stream into a char stream, and then `br` reads that char stream — which is what we want! After this statement executes, `br` is a character-based stream that is linked to the console through `System.in`.

Writer Classes

OutputStreamWriter Class

Extends `Writer`. Used to write to character streams. The `OutputStreamWriter` class implements `AutoCloseable` so we can use the try-with-resources type exception handling. There are only four public methods in the `OutputStreamWriter` class: `close()`, `flush()`, `getEncoding()`, and `write()`.

The `write()` method has three overloaded versions:

- `write(int a)` — writes a single character to the character stream. The character being written is contained in the 16 lower bits of the 'char' integer value; the remaining 16 higher bits are ignored by the method.
- `write(String str)`
- `write(char cbuf[])`

FileWriter Class

Extends `OutputStreamWriter`. The purpose of the `FileWriter` class is to simply write character-based files. The `FileWriter` class implements `AutoCloseable` so we can use the try-with-resources type exception handling. The `FileWriter` class does not define or override any public methods, so it inherits all of its methods from its superclass `OutputStreamWriter`.

Object → Writer → OutputStreamWriter → FileWriter

BufferedWriter Class

The `BufferedWriter` class is used to write text to a character stream. It has three overloaded versions of the `write()` method. The `BufferedWriter` class introduces a method named `newLine()`, which means you will not have to hardcode `"\r\n"` into your output stream anymore. The `BufferedWriter` class implements `AutoCloseable` so we can use the `try-with-resources` type exception handling.

File Class

Consider this Windows hard-coded path represented as a string: `"c:\\Java\\BW\\Sample.txt"`. If a user attempted to run your program on a Linux or UNIX OS, your program would fail miserably. That is because the directory separator is `'/'` in UNIX, as opposed to `'\\'` in Windows. The `File` class provides us with several tools to dynamically create our directory and file structure.

Constructors:

```
File(File parent, String child)
File(String pathname) // not very useful for multi-platform
File(String parent, String child)
File(URI uri)
```